- Hybrid model training

Example:

```
import pennylane as qml

dev = qml.device("default.qubit", wires=1)

@qml.qnode(dev)
def circuit(theta):
    qml.RY(theta, wires=0)
    return qml.expval(qml.PauliZ(0))
```

## 6. TRAINING PROCESS

The hybrid training loop:

1. Input data $x$
2. Prepare quantum state
3. Measure output
4. Compute loss
5. Update parameters using classical optimizer

Mathematically:

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}$$

## FINAL CONNECTION

Quantum Machine Learning unifies:

1. Quantum state representation
2. Parametrized circuit design
3. Classical optimization
4. Data-driven learning

It extends classical machine learning into quantum Hilbert spaces.

## STUDY ROADMAP

**Phase 1 — Foundations**

- Parametrized quantum circuits
- Measurement theory

**Phase 2 — Core Models**

- QNNs
- VQE
- QAOA

**Phase 3 — Integration**

- Qiskit Machine Learning
- PennyLane AI

**Phase 4 — Training**

- Loss functions
- Optimization methods

**Phase 5 — Applications**

- Classification
- Optimization
- Quantum simulation

# Chapter 7

# DIFFERENTIALS (ADVANCED AND DESIRABLE SKILLS)

This block focuses on advanced competencies that enhance performance in machine learning, generative AI, and large-scale experimentation environments.

**These skills are not strictly required, but they significantly increase efficiency, scalability, and impact in real-world AI systems**

## 1. ADVANCED GENERATIVE AI

### 1.1 Reinforcement Learning from Human Feedback (RLHF)

RLHF is a technique used to align machine learning models with human preferences.

**Pipeline Structure**

1. Pretrain a base model
2. Collect human feedback
3. Train a reward model
4. Optimize the policy using reinforcement learning

### Mathematical Formulation

The objective is to maximize expected reward:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \pi_\theta}[R(x)]$$

where:

- $\pi_\theta$ is the model policy
- $R(x)$ is the learned reward function

Policy update (example using policy gradient):

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(x) \cdot R(x)\right]$$

## 1.2 Prompt Engineering and Dataset Design

### Prompt Engineering

- Designing structured inputs to guide model outputs
- Controlling behavior via instructions and context
- Few-shot and zero-shot prompting

### Dataset Engineering

- Data curation and cleaning
- Labeling and annotation strategies
- Balancing and augmentation

Example of structured prompt:

```
prompt = """
Task: Classify the sentiment of the sentence.
Input: The movie was excellent.
Output:
"""
```

# 2. INFRASTRUCTURE

## 2.1 GPU Computing

GPU computing accelerates large-scale numerical operations:

- Parallel processing of tensors
- Efficient training of deep neural networks
- Support via CUDA and specialized libraries

Example:

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
x = torch.randn(1000, 1000).to(device)
y = torch.matmul(x, x)
```

## 2.2 Experimental Pipelines

Structured workflows for running experiments:

- Data preprocessing
- Model training
- Evaluation and logging
- Reproducibility

Pipeline abstraction:

$$\text{Data} \rightarrow \text{Model} \rightarrow \text{Training} \rightarrow \text{Evaluation}$$

# 3. MLOPS

## 3.1 Overview

MLOps (Machine Learning Operations) focuses on managing the lifecycle of machine learning models.

## 3.2 MLflow

- Experiment tracking
- Model versioning
- Deployment support

Example:

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("lr", 0.01)
    mlflow.log_metric("loss", 0.25)
```

## 3.3 Weights & Biases

- Real-time experiment tracking
- Visualization dashboards
- Hyperparameter tuning

Example:

```
import wandb

wandb.init(project="example")
wandb.log({"loss": 0.25})
```

### 3.4 DVC (Data Version Control)

- Versioning datasets and models
- Reproducible pipelines
- Integration with Git

Example:

```
# Command-line usage (conceptual)
# dvc init
# dvc add data.csv
# git add data.csv.dvc
```

## FINAL CONNECTION

These advanced skills enhance:

1. **Model Alignment:** RLHF and prompt engineering
2. **Scalability:** GPU computing and pipelines
3. **Reproducibility:** MLOps tools
4. **Deployment Readiness:** experiment tracking and versioning

## STUDY ROADMAP

### Phase 1 — Generative AI

- RLHF fundamentals
- Prompt engineering techniques

**Phase 2 — Infrastructure**

- GPU acceleration
- Experiment pipelines

**Phase 3 — MLOps**

- MLflow
- Weights & Biases
- DVC

**Phase 4 — Integration**

- End-to-end ML workflows
- Scalable and reproducible systems

# Chapter 8

# SCIENTIFIC PRODUCTION

This block focuses on the tools and practices required to document, share, and reproduce scientific work in computational science, physics, and machine learning.

**Scientific production requires clear communication, version control, and reproducibility of experiments**

## 1. JUPYTER NOTEBOOKS

### 1.1 Overview

Jupyter Notebooks provide an interactive environment that combines code, text, and visualizations.

### 1.2 Key Features

- Executable code cells
- Markdown for documentation
- Inline visualization
- Step-by-step experimentation

### 1.3 Example

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y)
plt.show()
```

### 1.4 Applications

- Data analysis
- Prototyping models
- Educational material

## 2. LATEX

### 2.1 Overview

LaTeX is the standard typesetting system for scientific documents.

### 2.2 Mathematical Representation

$$E = mc^2$$

$$\int_a^b f(x)\,dx$$

### 2.3 Key Features

- High-quality mathematical typesetting
- Structured documents (sections, equations, references)
- Bibliography management

### 2.4 Applications

- Academic papers
- Reports and theses
- Technical documentation

## 3. GIT AND GITHUB

### 3.1 Version Control

Git enables tracking changes in code and documents over time.

### 3.2 Core Concepts

- Repository
- Commit history
- Branching and merging

### 3.3 Basic Workflow

```
# Conceptual commands (executed in terminal)
git init
git add .
git commit -m "Initial commit"
git push origin main
```

### 3.4 GitHub

- Remote repository hosting
- Collaboration via pull requests
- Issue tracking and project management

# 4. EXPERIMENTAL REPRODUCIBILITY

## 4.1 Definition

Reproducibility ensures that experiments can be repeated with consistent results.

## 4.2 Key Principles

- Fixed random seeds
- Versioned datasets
- Controlled environments
- Documented workflows

## 4.3 Mathematical Perspective

Given an experiment:

$$y = f(x, \theta)$$

Reproducibility requires that for fixed $x$ and $\theta$, the output $y$ remains consistent.

## 4.4 Example

```
import numpy as np


np.random.seed(42)
data = np.random.rand(5)
print(data)
```

# FINAL CONNECTION

Scientific production integrates:

1. **Exploration:** Jupyter Notebooks

2. **Documentation:** LaTeX
3. **Version Control:** Git/GitHub
4. **Reliability:** Reproducibility practices

## STUDY ROADMAP

**Phase 1 — Notebooks**

- Jupyter basics
- Combining code and text

**Phase 2 — Documentation**

- LaTeX structure
- Mathematical writing

**Phase 3 — Version Control**

- Git fundamentals
- GitHub collaboration

**Phase 4 — Reproducibility**

- Experiment tracking
- Environment management

**Phase 5 — Integration**

- End-to-end scientific workflows
- Research project organization